Práctica 3.

Librerías de ayuda para la implementación de la POO: *LOMBOK*

GitHut - Target

Manuela Planelles - 1º DAW - IES Mutxamel

INTRODUCCIÓN

En esta práctica utilizaremos Project Lombok, una librería de Java que elimina el código repetitivo como getters, setters y constructores. El objetivo es crear un sistema de gestión escolar para manejar estudiantes y cursos de forma eficiente.

Con Lombok, reduciremos la cantidad de código escrito, logrando clases más limpias y fáciles de leer. Además, aplicaremos conceptos clave como:

· Validación de datos y control de valores nulos.

· Detección de duplicados para evitar registros repetidos.

· Modelado visual mediante diagramas de clases en PlantUML.

Todo el proceso se realizará siguiendo las buenas prácticas de la programación orientada a objetos.

OBJETIVO DE LA PRACTICA

El desarrollo de este sistema de gestión se enfoca en alcanzar las siguientes metas técnicas:

· Gestión de Dependencias: Configurar Maven y el archivo pom.xml para integrar la librería Lombok.

· Automatización: Aplicar anotaciones como @Getter, @Setter y @AllArgsConstructor para eliminar código manual.

· Seguridad de Datos: Implementar @NonNull y validaciones para bloquear valores nulos en las listas.

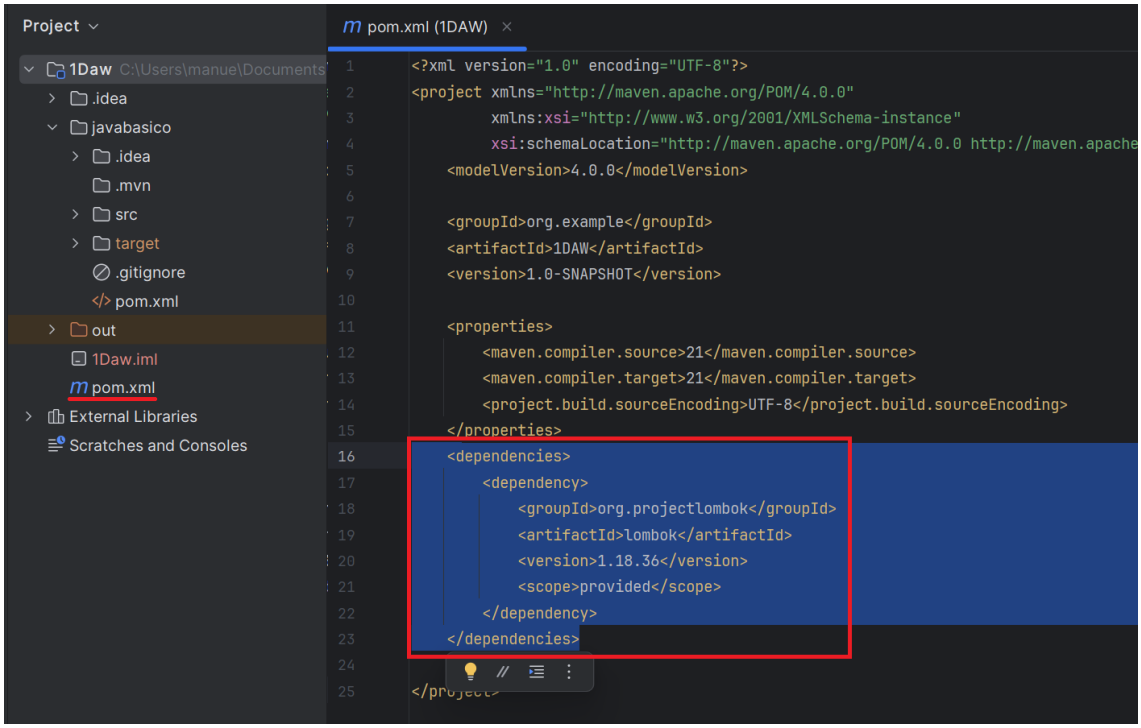
· Reglas de Negocio: Programar la detección de cursos duplicados comparando nombre y horas.

· Inmutabilidad: Garantizar que ciertos datos sean permanentes tras su creación usando final.

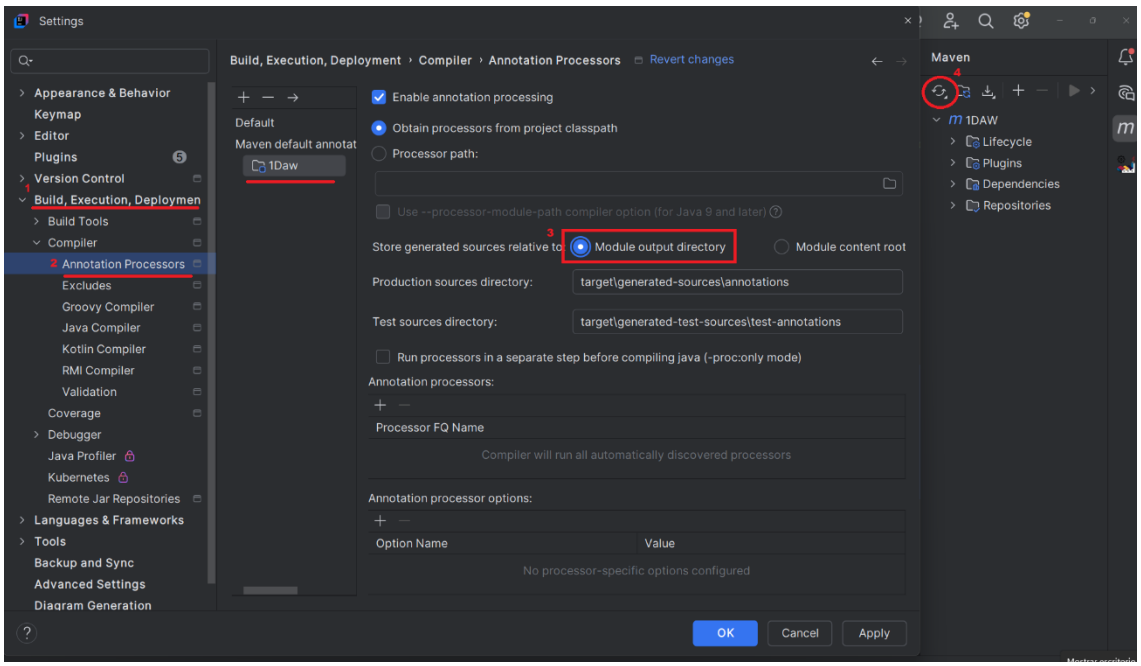
· Pruebas de Error: Crear un entorno de ejecución que capture excepciones y asegure la estabilidad del sistema.

INSTALACIÓN

PASO 1: Configurar el Proyecto Maven. Agrego el texto indicado de la practica en el archivo pom.xml



PASO 2: Configurar IntelliJ para Lombok. Actualizamos Maven, Habilitamos “Annotation Processors” e instalamos el Plugin “Lombok”.



DISEÑO E IMPLEMENTACIÓN DEL CODIGO

PASO 3: Crear la Clase Curso.

Clase Estudiante:

Creamos los atributos nombre, edad y curso. Indicamos encima de Nombre la anotación de Lombok para ello → @NotNull , para indicar que el nombre no puede estar vacío y es obligatorio.

También se indican el resto de las anotaciones, para los métodos get, set, to string y el constructor para los objetos.

pom.xml (1DAW) | Estudiante.java | Curso.java

```java
package practicas.LombokInstituto;
import lombok.*;

@NoArgsConstructor @AllArgsConstructor @RequiredArgsConstructor  no usages  new *
@Getter @Setter
@ToString
public class Estudiante {

    @NonNull
    private String nombre;
    private int edad;
    private Curso curso;
}
```

*Nota: se crea la clase Curso vacía de momento, para relacionar el atributo con la clase.

Clase Curso:

Hacemos el mismo proceso, creamos atributos y las anotaciones de “Lonbok”

pom.xml (1DAW) | Estudiante.java | LombokInstituto\Curso.java

```java
package practicas.LombokInstituto;
import lombok.*;

@NoArgsConstructor @AllArgsConstructor @RequiredArgsConstructor  1 usage  new *
@Getter @Setter
@ToString
public class Curso {

    @NonNull
    private String nombre;
    private int horas;
}
```

Clase Instituto:

Creamos los atributos, el constructor y el set lo controlamos de forma manual para poder modificar, así que solo pondremos el Gettter y el toString, agregamos @Setter para la población.

```
m pom.xml (1DAW)    Estudiante.java    Curso.java    Instituto.java ×
1   package practicas.LombokInstituto;
2   import lombok.Getter;
3   import lombok.NonNull;
4   import lombok.Setter;
5   import lombok.ToString;
6   import java.util.ArrayList;
7
8   @Getter @ToString  no usages  new *
9   public class Instituto {
10
11      @NonNull
12      private String nombre;
13      @Setter
14      private String poblacion;
15
16      private ArrayList<Estudiante> listaEstudiante;
17      private ArrayList<Curso> listaCurso;
```

Seguimos en esta clase, y pasamos a crear el constructor, debe recibir el nombre como parámetro, si el nombre es null, que indique “Instituto sin nombre” e inicializar las listasCurso como una Array vacío

```
public Instituto(String nombre) {  no usages  MANUELA\manue *
    this.nombre = Objects.requireNonNullElse(nombre, defaultObj: "Instituto sin nombre");
    this.listaEstudiantes = new ArrayList<>();
    this.listaCursos = new ArrayList<>();
}
```

Clase AppCurso:

Creamos la clase de la app para hacer la primera comprobación y funciona correctamente.

```java
package practicas.LombokInstituto;

public class AppCurso {
    public static void main(String[] args) {

        Instituto instSinNombre = new Instituto( nombre: null);
        System.out.println("Nombre asignado: " + instSinNombre.getNombre());
    }
}
```

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Commun
Nombre asignado: Instituto sin nombre

Process finished with exit code 0
```

Creamos un Método para agregar Estudiantes y evitar Null, usamos if para la comprobación con el mensaje de control. Y comprobamos.

```java
public class Instituto {
    public Instituto(String nombre) {
        this.listaEstudiantes = new ArrayList<>();
        this.listaCursos = new ArrayList<>();
    }
    public void agregarEstudiante(Estudiante estudiante) {
        if (estudiante != null) {
            this.listaEstudiantes.add(estudiante);
        } else {
            System.out.println("No se puede agregar un estudiante nulo.");
        }
    }
```

```java
package practicas.LombokInstituto;

public class AppCurso {
    public static void main(String[] args) {

        Instituto instSinNombre = new Instituto( nombre: null);
        System.out.println("Nombre asignado: " + instSinNombre.getNombre());
        instSinNombre.agregarEstudiante(null);

    }
}
```

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.2.2\lib\idea_rt.jar=50584" -Dfile.en
Nombre asignado: Instituto sin nombre
No se puede agregar un estudiante nulo.

Process finished with exit code 0
```

Para la última parte evitar la inserción de valor null y la duplicación de cursos, generar equals()

```
@Override  new *
public boolean equals(Object otroCurso) {
    if (this == otroCurso) return true;
    if (otroCurso == null || getClass() != otroCurso.getClass()) return false;
    Curso curso = (Curso) otroCurso;
    return horas == curso.horas && nombre.equals(curso.nombre);
}
```

Para comprobar si el curso actual o el que recibe son lo mismo.

```
if (this == otroCurso) return true;
```

Para descartar si no existe o si aparece alguna clase distinta a la de Clase

```
if (otroCurso == null || getClass() != otroCurso.getClass()) return false;
```

Transformamos el genérico en tipo curso para acceder a los datos.

```
Curso curso = (Curso) otroCurso;
```

Para indicar si el valor es idéntico.

```
return horas == curso.horas && nombre.equals(curso.nombre);
```

y hashCode() en Curso para meter en la lista

```
@Override  new *
public int hashCode() {
    return java.util.Objects.hash(nombre, horas);
}
```

Y en la clase Instituto terminamos creando el método agregarCurso() para llamar las comprobaciones y que salte el mensaje.

```
32      public void agregarCurso(Curso curso) {  no usages  MANUELA\manue
33          if (curso == null) {
34              System.out.println("No se puede agregar un curso nulo.");
35              return;
            }
37          if (!listaCursos.contains(curso)) {
38              this.listaCursos.add(curso);
39          } else {
40              System.out.println("El curso ya existe en el instituto.");
41          }
```

PLAN DE PRUEBAS

1. Comprobar la Clase Estudiante. Esto debería lanzar una excepción porque el nombre es @NonNull

```java
package practicas.LombokInstituto;

public class AppCurso {
    public static void main(String[] args) {
        try {
            Estudiante estudianteError = new Estudiante(nombre: null, edad: 20, curso: null);
        } catch (NullPointerException e) {
            System.out.println("Comprobación Estudiante: Bloqueada la creación de un estudiante sin nombre.");
        }
```

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Editio
Comprobación Estudiante: Bloqueada la creación de un estudiante sin nombre.

Process finished with exit code 0
```

2. Comprobar la Clase Curso e Identidad. Si el equals está bien hecho, esto dirá 'true'

```java
package practicas.LombokInstituto;

public class AppCurso {
    public static void main(String[] args) {

        Curso curso1 = new Curso(nombre: "Java", horas: 100);
        Curso curso2 = new Curso(nombre: "Java", horas: 100);
        System.out.println("¿Son el mismo curso?: " + curso1.equals(curso2));
    }
}
```

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\I
¿Son el mismo curso?: true

Process finished with exit code 0
```

3. Comprobar la agregación en el Instituto.

· Añadir un curso normal.

· Intentar añadir el mismo curso otra vez.

· Debería salir el mensaje: "El curso ya existe en el instituto."

· Intentar añadir un curso nulo.

· Debería salir el mensaje: "No se puede agregar un curso nulo.

```java
public class AppCurso {
    public static void main(String[] args) {

        Instituto miInstituto = new Instituto("IES Mutxamel");
        Curso cursoJava = new Curso("Java", 100);

        miInstituto.agregarCurso(cursoJava);

        miInstituto.agregarCurso(cursoJava);

        miInstituto.agregarCurso(null);
    }
}
```

```
Run  AppCurso
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files
El curso ya existe en el instituto.
No se puede agregar un curso nulo.

Process finished with exit code 0
```

SALIDA ESPERADA

Al ejecutar el programa de prueba, la consola debe mostrar la gestión de errores y el estado final de las listas:

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.2.2\lib\idea_rt.jar=58263" -Dfile
Error: No se puede crear un estudiante con nombre nulo.
No se puede agregar un estudiante nulo.
No se puede agregar un curso nulo.
Cursos disponibles:
[Curso(nombre=Java, horas=100), Curso(nombre=Python, horas=70)]
Estudiantes registrados:
[Estudiante(nombre=Carlos, edad=20, curso=Curso(nombre=Java, horas=100)), Estudiante(nombre=Ana, edad=22, curso=Curso(nombre=Python, horas=70))]

Process finished with exit code 0
```

CONCLUSIÓN

El uso de Lombok optimiza el desarrollo en Java al reducir drásticamente el código manual, permitiendo al programador centrarse en la lógica de negocio y validaciones. La combinación con validaciones de nulos y duplicados garantiza la integridad de los datos en el sistema.